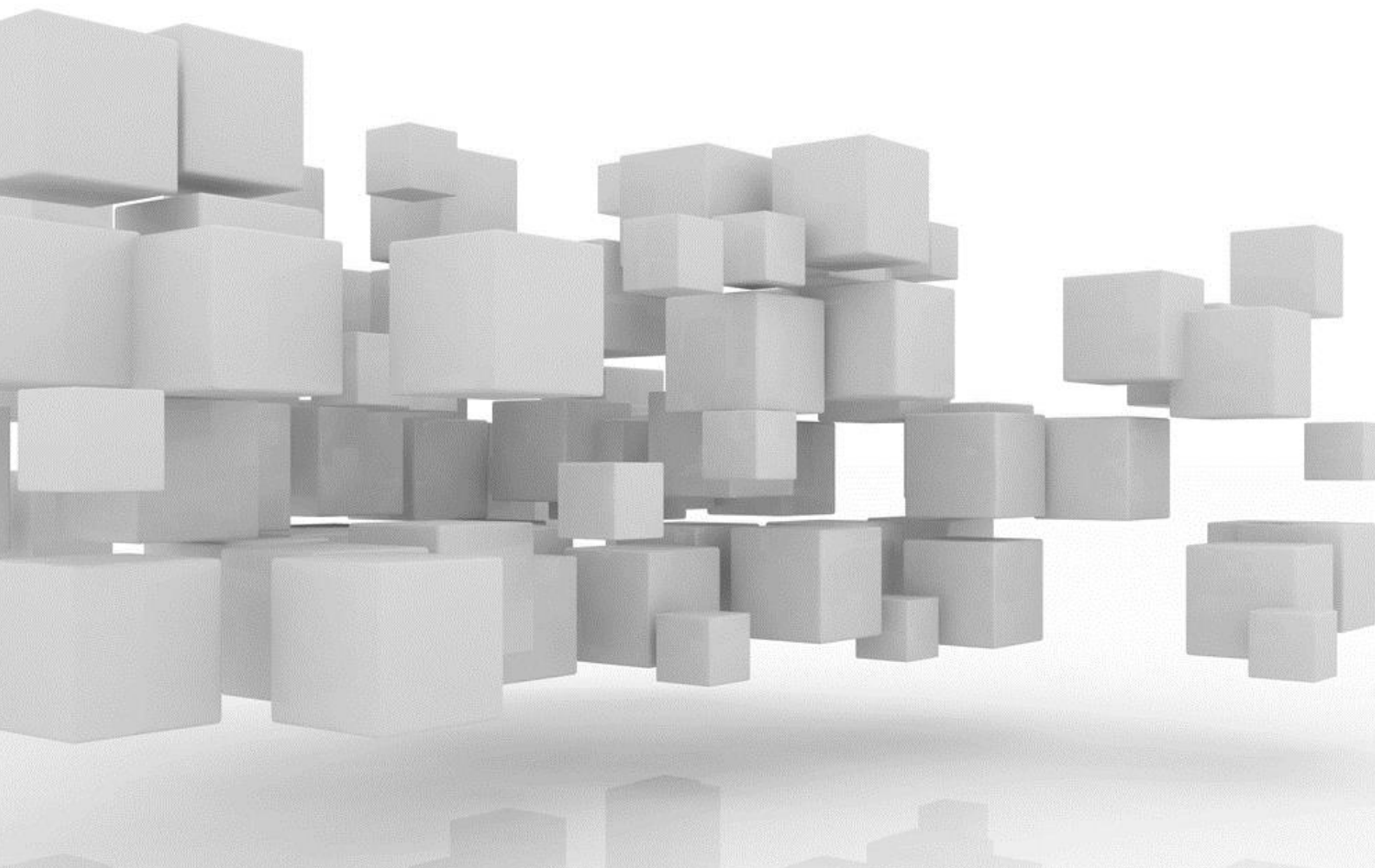


CycurLIB 3.5
User Manual



Copyright

The data in this document may not be altered or amended without special notification from ESCRYPT GmbH. ESCRYPT GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ESCRYPT GmbH.

© Copyright 2017 ESCRYPT GmbH, Bochum

The names and designations used in this document are trademarks or brands belonging to the respective owners.

Contents

1	Introduction.....	7
2	Installation.....	8
2.1	Directory Structure.....	8
2.2	Configuration	8
2.3	Tests	10
3	Compilation	11
4	Modules	12
4.1	Module dependency	12
4.2	Internal Modules	12
4.3	Return Values	12
5	Time-Sliced Implementations	14
6	Usage of a Random Number Generator	15
7	Module description	16
7.1	AES.....	16
7.2	Blowfish	18
7.3	ChaCha20	19
7.4	ChaCha20-Poly1305.....	19
7.5	CMAC-AES	19
7.6	CMAC-TDEA.....	19
7.7	CRC-32	20
7.8	DER	20
7.9	Triple-DES	21
7.10	Diffie-Hellman (DH)	22
7.11	ECC.....	23
7.12	ECC KDF.....	24
7.13	ECIES	25
7.14	EdDsa / Ed25519	25
7.15	Hash_DRBG	26
7.16	HMAC-RIPEMD-160.....	27
7.17	HMAC-SHA-1	27
7.18	HMAC-SHA-256	28
7.19	HMAC-SHA-512	28
7.20	PKCS#1 RSASSA V1_5	28
7.21	PKCS#1 RSASSA PSS	29
7.22	PKCS#1 RSAES V1_5.....	30
7.23	PKCS#1 RSAES OAEP.....	30
7.24	Poly1305	31
7.25	RIPEMD-160.....	31
7.26	RSA	32
7.27	SHA-1	33
7.28	SHA2-224 and SHA2-256	33
7.29	SHA2-384 and SHA2-512	34
7.30	SHA3	34
7.31	SHE-MP	34
7.32	Whirlpool.....	34
7.33	X.509	35

8	References.....	39
9	ESCRYPT Contact Addresses.....	41

Figures

Figure 1: Module dependencies. Modules without any dependencies are not shown.....	13
Figure 2: Sequence diagram of calling a non-sliced variant of RSA/ECC.	14
Figure 3: Sequence diagram of calling a sliced variant of RSA/ECC.	14

Tables

Table 1: Types / symbols defined in <code>cycurlib_config.h</code>	9
Table 2: General configuration options in <code>cycurlib_config.h</code>	10
Table 3: Configuration options for tests	10
Table 4: Make targets	11
Table 5: Make variables	11
Table 6: AES modes of operation.....	16
Table 7: AES Configuration	17
Table 8: Associated files for AES	18
Table 9: Associated files for Blowfish	19
Table 10: Associated files for ChaCha20	19
Table 11: Associated files for ChaCha20-Poly1305	19
Table 12: Associated files for CMAC-AES	19
Table 13: Associated files for CMAC-TDEA.....	20
Table 14: CRC-32 configuration	20
Table 15: Associated files for CRC-32.....	20
Table 16: Associated files for DER.....	21
Table 17: DES configuration	21
Table 18: Associated files for DES/3DES	22
Table 19: Configuration options for the DH module	22
Table 20: Associated files for Diffie-Hellmann.....	22
Table 21: ECC configuration	23
Table 22: Configuration of supported elliptic curves	24
Table 23: Associated files for ECC	24
Table 24: Associated files for ECC KDF	25
Table 25: ECIES configuration.....	25
Table 26: Associated files for ECIES	25
Table 27: Associated files for EdDSA	26
Table 28: Hash_DRBG configuration.....	27
Table 29: Associated files for Hash_DRBG	27
Table 30: Associated files HMAC-RIPEMD-160.....	27

Table 31: Associated files for HMAC-SHA-1.....	28
Table 32: Associated files for HMAC-SHA-256	28
Table 33: Associated files for HMAC-SHA-512	28
Table 34: PKCS#1 RSASSA V1_5 Configuration.....	29
Table 35: Associated files for PKCS#1 RSASSA V1_5.....	29
Table 36: PKCS#1 RSASSA PSS configuration	30
Table 37: Associated files for PKCS#1 RSASSA PSS.....	30
Table 38: Associated files for PKCS#1 RSAES V1_5	30
Table 39: PKCS#1 RSAES OAEP configuration	31
Table 40: Associated files for PKCS#1 RSAES OAEP	31
Table 41: Associated files for Poly1305.....	31
Table 42: Associated files for RIPEMD-160.....	32
Table 43: RSA configuration	33
Table 44: Associated files for RSA.....	33
Table 45: Associated files for SHA-1.....	33
Table 46: Associated files for SHA2-256	34
Table 47: Associated files for SHA2-512	34
Table 48: Associated files for SHA3	34
Table 49: Associated files for SHE-MP	34
Table 50: Whirlpool configuration	35
Table 51: Associated files for Whirlpool.....	35
Table 52: X.509 configuration.....	37
Table 53: Associated files for X.509	38

1 Introduction

ESCRYPT's cryptographic library *CycurLIB* is a collection of common cryptographic algorithms. CycurLIB is designed to be used on embedded systems where resources are particularly limited.

The library meets the following design criteria:

- Minimized ROM size
- Minimized RAM usage
- Compliance with MISRA-C:2012
- No dependencies on external libraries (including the C standard library)
- No dependencies on a particular microcontroller or operating system (e.g. independence of endianness, no assembly code)
- No dynamic memory allocation
- Fully re-entrant
- Easy to integrate in existing systems

2 Installation

The CycurLIB will be delivered in a ZIP file format. When the files are uncompressed, the user receives the destination folder `cycurlib`.

This folder contains the CycurLIB, tests of the CycurLIB as well as documentation.

2.1 Directory Structure

<code>cycurlib</code>	Root directory. Contains makefiles and configuration
— <code>doc</code>	Documentation of CycurLIB
— <code>lib</code>	Source code of CycurLIB
— <code>bin</code>	Contains binary of library
— <code>inc</code>	Include files (*.h)
— <code>src</code>	Source files (*.c)
— <code>tests</code>	Tests of CycurLIB
— <code>bin</code>	Binary of tests in library form
— <code>inc</code>	Include files (*.h)
— <code>src</code>	Source files (*.c)

2.2 Configuration

CycurLIB can be configured at compile-time via defining preprocessor symbols. The value of each configuration option can be set by changing the definition in the corresponding header file directly. The configuration options are located in the configuration section of a header file as indicated by the following comment:

```

/*****
 * 3. Configuration
 *****/

```

For example, to change the maximum key size of AES from 128 bits to 256 bits, the line

```
#define EscAes_MAX_KEY_BITS 256U
```

In the header file `_aes.h` needs to be changed into

```
#define EscAes_MAX_KEY_BITS 256U
```

An alternate way to change these options is to define the symbols at the command line when the compiler is invoked. For example, to set the AES maximum key size to 256 bits as shown above, a gcc-based compiler could be invoked with the switch

```
-DEscAes_MAX_KEY_BITS=256U
```

Note: If the configuration symbols are defined via the command line then care must be taken that both the CycurLIB and the source code using the CycurLIB are compiled with the same configuration settings.

Most options are flags which can be set to 0 (disable) or 1 (enable). Furthermore, a lot of key sizes are configured at compile time as shown in the example above. Central configuration options which affect the entire library are located in the header file `cyclib_config.h`. Algorithm-related options are located in the header file of the algorithm's module (see section 0).

Table 1 shows all primitive types used by the Cyclib. Their specific types are defined in the file `cyclib_config.h`. It is important that these types match the expected size on the target microcontroller. For example, `Esc_UINT32` should be an unsigned integral type which has a size of 32 bits. By default, the user does not need to adapt these type definitions, because they are defined using the system file `<stdint.h>` that was introduced with the C99 standard (see also option `Esc_USE_STDLIB_DECLARATIONS` in Table 2).

Type / Symbol	Description	Default
<code>Esc_UINT64</code>	64 Bit unsigned integer (only if <code>Esc_HAS_INT64</code> is 1)	unsigned long long
<code>Esc_SINT64</code>	64 Bit signed integer (only if <code>Esc_HAS_INT64</code> is 1)	signed long long
<code>Esc_UINT32</code>	32 Bit unsigned integer	unsigned long
<code>Esc_SINT32</code>	32 Bit signed integer	signed long
<code>Esc_UINT16</code>	16 Bit unsigned integer	unsigned short
<code>Esc_SINT16</code>	16 Bit signed integer	signed short
<code>Esc_UINT8</code>	8 Bit unsigned integer	unsigned char
<code>Esc_SINT8</code>	8 Bit signed integer	signed char
<code>Esc_CHAR</code>	Character of ANSI-C string (only used in tests)	char
<code>Esc_BOOL</code>	Boolean data type that can only hold the values TRUE or FALSE. Variables of this data type fulfill the expected "Boolean equations" (for example: (TRUE or FALSE) == TRUE)	<code>Esc_UINT8</code>
<code>Esc_ERROR</code>	Integral data type to hold return codes	<code>Esc_UINT8</code>
<code>TRUE</code>	<code>Esc_BOOL</code> constant value TRUE	1
<code>FALSE</code>	<code>Esc_BOOL</code> constant value FALSE	0

Table 1: Types / symbols defined in `cyclib_config.h`

Table 2 shows further general configurations options that are located in the file `cycurlib_config.h`.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>Esc_USE_STDLIB_DECLARATIONS</code>	If this option is enabled, all basic types are defined using the system file <code><stdint.h></code> . Otherwise, the types are directly defined to plain C types in the file.	0 / 1	1
<code>Esc_HAS_INT64</code>	Enabling <code>Esc_HAS_INT64</code> allocates <code>Esc_UINT64</code> and <code>Esc_SINT64</code> data types. The application of these data types enables many systems to significantly speedup some modules.	0 / 1	0
<code>Esc_OPTIMIZE_SPEED</code>	Enabling <code>Esc_OPTIMIZE_SPEED</code> leads to speed optimized code, with higher ROM and RAM requirements.	0 / 1	0
<code>Esc_NDEBUG</code>	Enabling <code>Esc_NDEBUG</code> disables assertions.	0 / 1	0

Table 2: General configuration options in `cycurlib_config.h`

2.3 Tests

The folder `tests` contains a second library which performs self-tests under the use of CyclicLIB. The self-tests were designed to meet the same criteria as the actual CyclicLIB code (cf. section 1).

The function `EscTst_Perform()` is the entry point to execute all tests. The function is declared in the header file `selftest.h` which also contains the configuration options listed in Table 3.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscTst_ENABLE_LOGGING</code>	Enables the debug outputs for the CyclicLIB selftest. The default implementation for the output violates MISRA by using <code>printf()</code> of <code>stdio.h</code> .	0 / 1	0

Table 3: Configuration options for tests

3 Compilation

The CycurLIB can be compiled using the included makefile which is located in the folder `cycurlib`. The makefile is written for compilation under Cygwin or Linux and assumes the following toolchain:

- GNU make (tested with version 4.1)
- GCC-based compiler (tested with GCC version 5.4)
- GNU binutils (tested with version 2.25.2)

Please note that the versions listed above are not necessarily required and older versions of the tools will most probably work as well. The targets provided by the makefile are listed in Table 4.

Target	Description
all	Build the targets „lib“ and „test“.
lib	Build the CycurLIB as statically linked library. The output is located in the file <code>cycurlib/lib/bin/libcycurlib.a</code> .
tests	Build the test library for the current configuration of the CycurLIB. The output is located in the file <code>cycurlib/tests/bin/libcycurlibtest.a</code> .
clean	Delete all binaries and object files.
help	Display an overview of the existing make targets.

Table 4: Make targets

The makefile can be customized to use a different compiler and a different archive tool to create the static library. Also, flags passed to the compiler and to the archive tool can be customized in the makefile. The corresponding variables are shown in Table 5. The variables can also be overridden in the command line. For example, the following make invocation uses a GCC based compiler and archive tool for the ARM architecture:

```
make all CC=arm-elf-gcc AR=arm-elf-ar
```

Variable	Description
CC	Compiler to be used. Default: gcc
AR	Archive tool to be used. Default: ar
CFLAGS	Options passed to the compiler. Default: -Wall -Wextra -Wundef
ARFLAGS	Options passed to the archive tool. Default: r

Table 5: Make variables

4 Modules

The library is subdivided into modules, which can be removed as well as extracted individually.

In order to adopt, for instance, the module CRC-32 in an own project, it is sufficient to integrate the files `crc_32.c`, `crc_32.h`, `cyclib_config.h` and `cyclib_error.h` in an own development environment.

4.1 Module dependency

Some modules cannot be compiled and run capable by themselves – they require further modules of the Cyclib. The module dependencies, even of sub modules are shown in Figure 1. In the figure, the module “RANDOM” denotes any cryptographically strong pseudo-random number generator. For instance, Cyclib’s implementation (see section 7.14) of the Hash_DRBG could be used.

4.2 Internal Modules

Some modules are only intended for internal use. Their filename is prepended with an underscore, e.g. `_aes.c`. Using an internal module directly is not supported by the library.

4.3 Return Values

All Cyclib functions provide an error code of type `Esc_ERROR` which is defined as an unsigned 8-bit integer type by default (see also section 2.2). The error code `Esc_NO_ERROR` means that the function worked correctly. All other error codes mean that the corresponding function failed. A list of error codes can be found in the file `cyclib_error.h` or in the corresponding doxygen documentation.

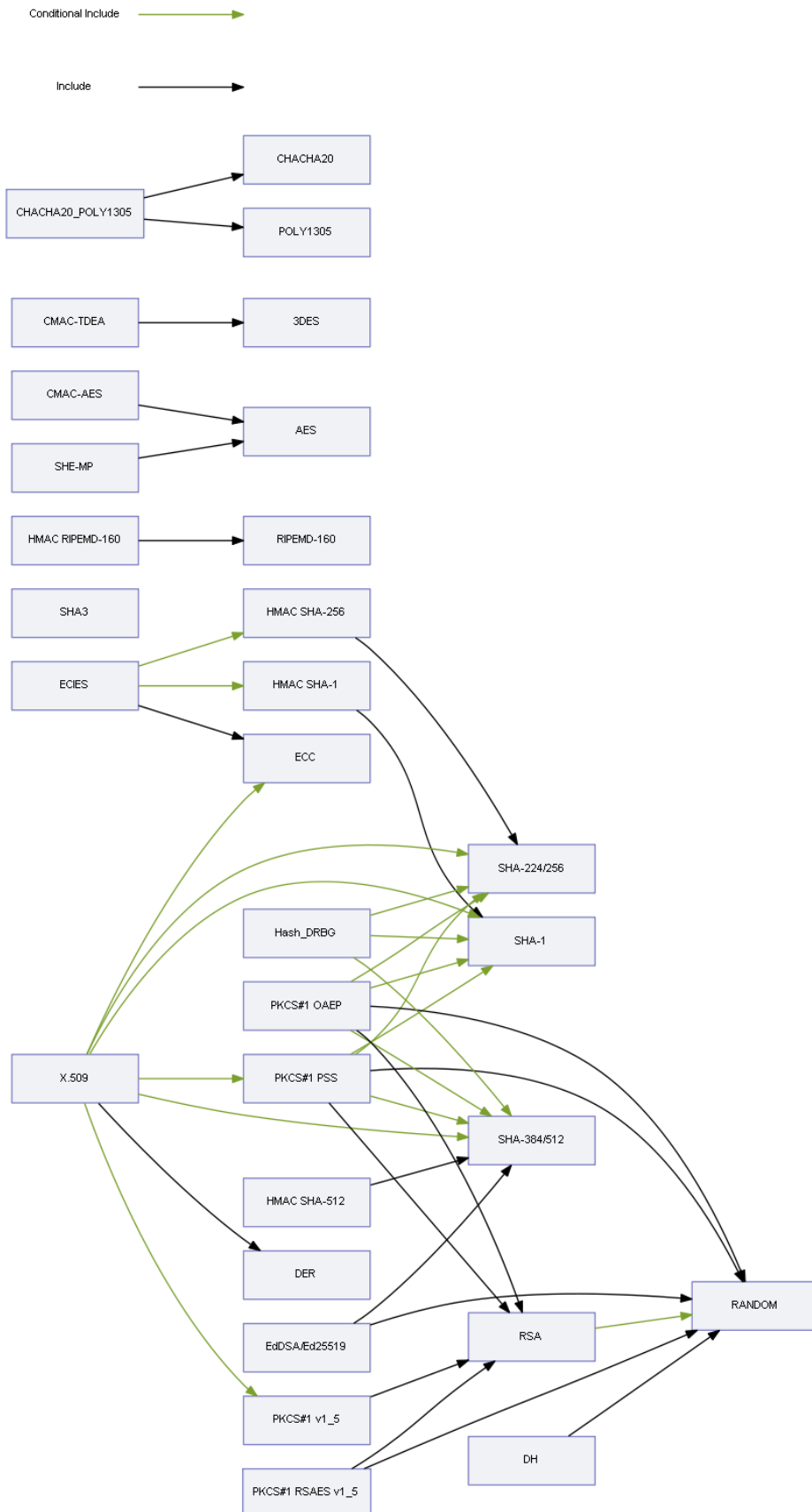


Figure 1: Module dependencies. Modules without any dependencies are not shown.

5 Time-Sliced Implementations

Asymmetric cryptographic functions such as signature generation with RSA or ECC are computationally intensive. This may be problematic if the environment of an embedded system does not allow long runtimes for a single function call. Typical scenarios might be:

- A watchdog must be called periodically
- The system must react quickly to periodic events, but no pre-emptive scheduler is available

To enable the usage of such algorithms even in these scenarios, the CycurLIB provides a so called *time-sliced* (or *sliced* for short) implementation for most RSA/ECC operations. Depending on the configuration, a non-sliced implementation can be used as well. The non-sliced variant simply computes the entire computation in a single call as shown in Figure 2.

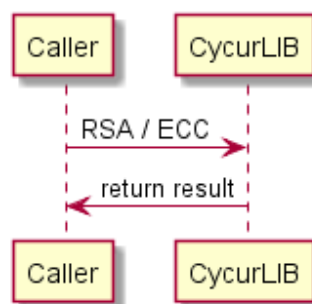


Figure 2: Sequence diagram of calling a non-sliced variant of RSA/ECC.

The sliced variant on the other hand quickly returns to the caller and signals by its return code that it must be called again. The caller may now chose to do something else before the cryptographic function is called again. Eventually, the function must be called until it finished either successfully or with a failure. This behavior is depicted in the sequence diagram shown in Figure 3.

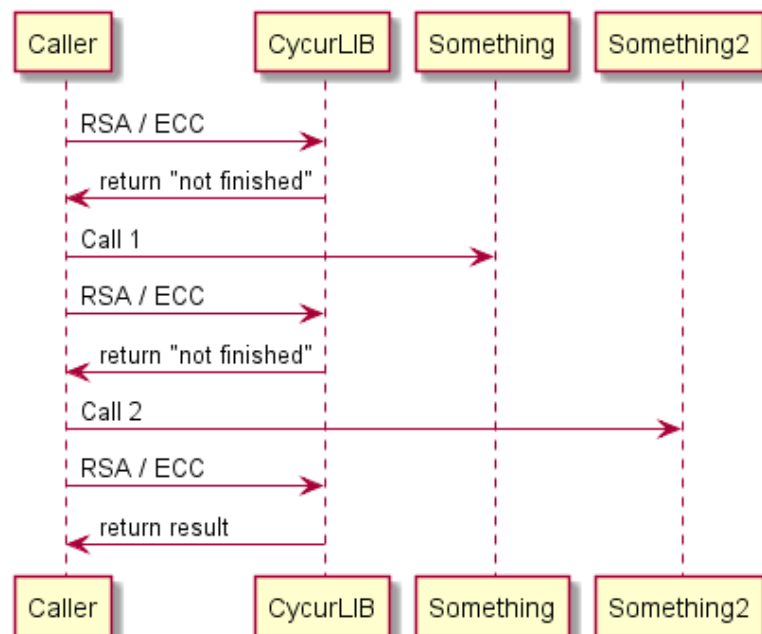


Figure 3: Sequence diagram of calling a sliced variant of RSA/ECC.

6 Usage of a Random Number Generator

Some cryptographic algorithms such as key generation or some signature schemes require random numbers. Generally, *random numbers* means that the data has a uniform distribution and each of the values of the data set is equally probable.

In the context of cryptography random numbers must also be non-predictable. That means that one neither can predict succeeding numbers of one random number generator nor compute preceding numbers. Such a random number generator is also called cryptographically secure pseudo-random number generator (*CSPRNG*).

All our functions which require a CSPRNG have two parameters. The first one for providing a pointer to the state or context of the random number generator. The second one is a function pointer which must point to the GetRandom function of the CSPRNG which must have the format as defined in random.h:

```
typedef Esc_ERROR (*EscRandom_GetRandom) (
    void* prngState,
    Esc_UINT8 rnd[],
    const Esc_UINT32 len );
```

Depending on the random number generator, it must be seeded before the first usage.

The CycurLIB implements a CSPRNG called Hash_DRBG, cf. Section 7.15, which provides a wrapper which fits to the defined interface in random.h. The Hash_DRBG must be seeded with a sufficient amount of entropy via the function *EscHasDrbg_Init()*.

An exemplary usage of our Hash_DRBG for the RSA key generation would be as follows:

```
#include "hash_drbg.h"

[...]

returnCode = EscHashDrbg_Init( &CsprngCtx, seed, nonce );

if ( returnCode == Esc_NO_ERROR )
{
    returnCode = EscRsa_KeyGeneration(
        EscHashDrbg_GetRandomWrapper,
        &CsprngCtx,
        &privKey,
        65537U,
        2048U,
        FALSE );
}
```

7 Module description

The individual modules of the CyscurLIB are briefly described in this section. For each module, all configuration options are explained. Also, security hints are given for some modules.

The associated files for each module are listed in a separate table.



Warning!

The following sections warn for legacy algorithms, which should not be used in new systems any more for security reasons.

These security hints cannot be guaranteed to be complete, because an algorithm that is considered secure today may be broken in the future! Additionally, parameters for each algorithm must be chosen carefully for the given security objectives!

7.1 AES

AES is a symmetric block cipher and can be used for encryption and decryption, and depending on the mode of operation also for symmetric authentication. The algorithm was implemented in accordance to FIPS-197 [AES]. The supported key lengths are 128, 192 and 256 bit and the supported modes are listed in Table 6.

Mode	Provides Encryption?	Provides Authentication?	Depends on Mode
ECB	Yes	No	-
CBC	Yes	No	-
CBC-MAC	No	Yes	CBC
CMAC (OMAC1)	Yes	Yes	-
CTR	Yes	No	-
EAX	Yes	Yes	CTR, CMAC-AES
GCM	Yes	Yes	-
Key-Wrap	Yes	No	ECB
CCM	Yes	Yes	-
XTS	Yes	No	-

Table 6: AES modes of operation

One of three variants can be configured: A fast one, a small one and a compromise between these two. The configuration options are shown in Table 7. Note that

`EscAes_USE_T_TABLES_SMALL` and `EscAes_USE_T_TABLES_FULL` cannot be set both to 1 at the same time. The configuration options are used for all modes and can be found in

the header file `_aes.h`. Additionally, a faster version of the GCM-mode can be activated by enabling `EscAesGcm_USE_TABLES` in the header file `aes_gcm.h`.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscAes_KEY_BITS</code>	Maximum size of supported AES key length in bits. For instance, if 192 bits is configured then 128 and 192 bits are supported at runtime.	128, 192, 256	256
<code>EscAes_USE_T_TABLES_SMALL</code>	If set to 1, tables with a total size of 2 kb are used to speed-up encryption and decryption.	0 / 1	speed: 0 size: 0
<code>EscAes_USE_T_TABLES_FULL</code>	If set to 1, tables with a total size of 8 kb are used to speed-up encryption and decryption even more.	0 / 1	speed: 1 size: 0
<code>EscAesGcm_USE_TABLES</code>	If set to 1, the GCM-mode uses lookup tables to speed-up the GF(2 ¹²⁸)-multiplication significantly. The implementation uses two tables: One table of 4 kb size per key (stored in the context) and one key-independent table of 512 bytes size.	0 / 1	speed: 1 size: 0

Table 7: AES Configuration

<i>Lib/src</i>	<i>Lib/inc</i>
<code>_aes.c</code>	<code>_aes.h</code>
<i>Optional files (depending on your use case and configuration)</i>	
<code>_aes_dec.c</code>	
<code>_aes_enc.c</code>	
<code>aes_cbc.c</code>	<code>aes_cbc.h</code>
<code>aes_cbc_mac.c</code>	<code>aes_cbc_mac.h</code>
<code>aes_ccm.c</code>	<code>aes_ccm.h</code>
<code>aes_ctr.c</code>	<code>aes_ctr.h</code>
<code>aes_eax.c</code>	<code>aes_eax.h</code>
<code>aes_ecb.c</code>	<code>aes_ecb.h</code>
<code>aes_gcm.c</code>	<code>aes_gcm.h</code>
<code>aes_key_wrap.c</code>	<code>aes_key_wrap.h</code>
<code>aes_xts.c</code>	<code>aes_xts.h</code>

Table 8: Associated files for AES

7.2 Blowfish

This module implements the symmetric block cipher Blowfish according to [BLOWFISH]. The supported key lengths are in range between 32 and 448 Bit. The only supported mode is ECB. The key size can be selected at runtime.

<i>Lib/src</i>	<i>Lib/inc</i>
blowfish.c	blowfish.h

Table 9: Associated files for Blowfish

7.3 ChaCha20

This module implements the symmetric stream cipher ChaCha20 according to [RFC7539]. The supported key lengths are 128 and 256 Bit. The key size can be selected at runtime.

<i>Lib/src</i>	<i>Lib/inc</i>
chacha20.c	chacha20.h

Table 10: Associated files for ChaCha20

7.4 ChaCha20-Poly1305

This module implements the AEAD mode of the ChaCha20 stream cipher and the Poly1305 one-time authenticator in accordance with [RFC7539]. This mode requires a fixed key size of 256 Bit and a 96-bit nonce.

<i>Lib/src</i>	<i>Lib/inc</i>
chacha20_poly1305.c	chacha20_poly1305.h

Table 11: Associated files for ChaCha20-Poly1305

7.5 CMAC-AES

This module implements CMAC based on the AES in accordance to [CMAC].

Required module is AES with Encryption.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
cmac_aes.c	cmac_aes.h

Table 12: Associated files for CMAC-AES

7.6 CMAC-TDEA

This module implements CMAC based on the Triple-DES (TDEA) in accordance to [CMAC].

Required module is Triple-DES with the ECB mode.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
cmac_des3.c	cmac_des3.h

Table 13: Associated files for CMAC-TDEA

7.7 CRC-32

This module implements CRC-32 checksum calculation according to [CRC32]. A speed optimized and a size optimized variant can be used. Detailed configuration options are shown in Table 14.



Warning!

CRC32 is only a checksum algorithm and NOT a cryptographic hash function. CRC32 can NOT be used to fulfill any security objectives!

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
EscCrc32_INITIAL_VALUE	CRC initial value	Any unsigned 32-bit value	0xFFFFFFFFU
EscCrc32_QUOTIENT	Quotient polynomial, can only be configured in size optimized version (EscCrc32_USE_TABLE is defined to 0)	Any unsigned 32-bit value	0xEDB88320U
EscCrc32_USE_TABLE	Increases speed significantly on costs of one 1KB table	0 / 1	speed: 1 size: 0

Table 14: CRC-32 configuration

<i>Lib/src</i>	<i>Lib/inc</i>
crc32.c	crc32.h

Table 15: Associated files for CRC-32

7.8 DER

This module implements a parser for the Distinguished Encoding Rules (DER) defined in [

RFC8032 Edwards-Curve Digital Signature Algorithm (EdDSA)
<https://tools.ietf.org/html/rfc8032>

DER ITU-T x.690 (Distinguished Encoding Rules for ASN.1)
<http://www.itu.int/ITU-T/studygroups/com17/languages/X.690-0207.pdf>

]. The following ASN.1 [ASN] data types are supported by the parser:

- BIT_STRING
- BOOLEAN
- Generalized Time
- IA5String
- INTEGER
- NULL
- OBJECT_IDENTIFIER
- OCTET_STRING
- Printable String
- RELATIVE OID
- SET
- SEQUENCE
- UTC-Time
- VisibleString

<i>Lib/src</i>	<i>Lib/inc</i>
<code>der.c</code>	<code>der.h</code>

Table 16: Associated files for DER

7.9 Triple-DES

This module implements the symmetric cipher 3-key DES (TDEA) according to [DES]. The functions in the source file `_des.c` are intended for internal use only and should not be called directly by the user.

The supported modes for TDEA are ECB, CBC and OMAC1 (equivalent CMAC).

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscDes_USE_BIG_TABLES</code>	Increases speed significantly on costs of one lookup-table of 2 KB size.	0 / 1	speed: 1 size: 0

Table 17: DES configuration

<i>Lib/src</i>	<i>Lib/inc</i>
<code>_des.c</code>	<code>_des.h</code>
Optional files (depending on your use case and configuration)	
<code>des3_cbc.c</code>	<code>des3_cbc.h</code>
<code>des3_ecb.c</code>	<code>des3_ecb.h</code>

Table 18: Associated files for DES/3DES

7.10 Diffie-Hellman (DH)

This module implements the Diffie-Hellman key exchange. Generating key pairs and computing the shared secret are supported. Note that generation of the required domain parameters is currently not supported by the CysurLIB. This can be done with OpenSSL, for example. See <https://www.openssl.org/docs/apps/dhparam.html>

For the prime p , the values 1024, 2048 and 3072 bit are supported.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscDh_KEY_BITS_MAX</code>	Maximum size of prime modulus p .	1024, 2048 or 3072	1024
<code>EscDh_USE_SLIDING_WINDOW</code>	Enable to use the sliding window algorithm. It uses more RAM, but makes the computation faster.	0 / 1	speed: 1 size: 0
<code>EscDh_WINDOW_SIZE</code>	Size of the sliding window.	Any unsigned 32-bit value	4
<code>EscDh_GENERATION_ATTEMPTS</code>	Number of attempts to generate a key pair (depending on the random numbers this may fail).	Any unsigned 32-bit value	5

Table 19: Configuration options for the DH module

<i>Lib/src</i>	<i>Lib/inc</i>
<code>dh.c</code>	<code>dh.h</code>

Table 20: Associated files for Diffie-Hellmann

7.11 ECC

This module implements Elliptic Curve Cryptography in accordance to [SEC1], [SEC2], and [BRAINPOOL]. The following algorithms are included:

- ECDH (Key agreement with Diffie-Hellman)
- ECDSA (Signature verification and generation)
- Key pair generation
- Public key validation

ECC algorithms can be significantly accelerated by setting `Esc_HAS_INT64` to 1.

Algorithm optimizations can be configured in the file `ecc_cfg.h`. Refer to the comment in `ecc_cfg.h` for hints which of the algorithms should be used for your application.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscEcc_ECDH_ENABLED</code>	Enable Diffie Hellman key exchange	0 / 1	1
<code>EscEcc_ECDSA_ENABLED</code>	Enable DSA signature generation and verification	0 / 1	1
<code>EscEcc_USE_SLIDING_WINDOW</code>	Enable the Sliding Window algorithm	0 / 1	speed: 1 size: 0
<code>EscEcc_WINDOW_SIZE</code>	Set the window size for the Sliding Window algorithm	Any unsigned 16-bit value	4U
<code>EscEcc_USE_SHAMIRS_TRICK</code>	Enable the 'Shamir's trick' for signature verification	0 / 1	speed: 1 size: 0
<code>EscEcc_SLICING_ENABLED</code>	Enable to use the sliced variant.	0 / 1	0

Table 21: ECC configuration

As shown in Table 22, support for each curve implemented in the CypurLIB can be enabled or disabled in the configuration. At least one curve must be enabled.

<i>Preprocessor Symbol</i>	<i>Curve</i>	<i>Default</i>
<code>EscEcc_SECP_192_ENABLED</code>	secp192r1 / NIST P-192	1
<code>EscEcc_SECP_224_ENABLED</code>	secp224r1 / NIST P-224	0
<code>EscEcc_SECP_256_ENABLED</code>	secp256r1 / NIST P-256	0
<code>EscEcc_SECP_384_ENABLED</code>	secp384r1 / NIST P-384	0
<code>EscEcc_SECP_521_ENABLED</code>	secp521r1 / NIST P-521	0

EscEcc_BRAINPOOL_P160_ENABLED	brainpoolP160r1	0
EscEcc_BRAINPOOL_P192_ENABLED	brainpoolP192r1	0
EscEcc_BRAINPOOL_P224_ENABLED	brainpoolP224r1	0
EscEcc_BRAINPOOL_P256_ENABLED	brainpoolP256r1	0
EscEcc_BRAINPOOL_P320_ENABLED	brainpoolP320r1	0
EscEcc_BRAINPOOL_P384_ENABLED	brainpoolP384r1	0
EscEcc_BRAINPOOL_P512_ENABLED	brainpoolP512r1	0

Table 22: Configuration of supported elliptic curves

<i>Lib/src</i>	<i>Lib/inc</i>
ecc.c	ecc.h
_ptarithws.c	_ptarithws.h
_fearith.c	_fearith.h
	ecc_cfg.h
	eccarith.h
Optional files (depending on your use case and configuration)	
eccarith_s0.c	eccarith_s0.h
eccarith_s1.c	eccarith_s1.h

Table 23: Associated files for ECC

7.12 ECC KDF

This module implements an Elliptic Curve Cryptography key derivation function as referenced in [NIST 800-56]. The following algorithm is included:

- ECC Key Generate

<i>Lib/src</i>	<i>Lib/inc</i>
<code>ecc_kdf.c</code>	<code>ecc_kdf.h</code>

Table 24: Associated files for ECC KDF

7.13 ECIES

This module implements ECIES according to the IEEE Std 1363a [ECIES]. KDF ("KDF2") + XOR stream cipher encryption is used. The used MAC algorithm is the "HMAC".

Both, the DHAES and the Non-DHAES mode are supported. Note that in the Non-DHAES mode the scheme is malleable and is not secure against an adaptively chosen ciphertext attack (CCA2) (refer to [ECIES: D.5.3.3 - Note 1] for further information about this issue). Subject to the configuration, the modules ECC, HMAC-SHA-1, SHA-1, HMAC-SHA-256 and SHA-256 are required.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscEcies_DHAES_ENABLED</code>	Enable the DHAES Mode	0 / 1	1
<code>EscEcies_HASH_TYPE</code>	Determines the hash algorithm	<code>EscEcies_USE_SHA1</code> , <code>EscEcies_USE_SHA256</code>	<code>EscEcies_USE_SHA1</code>
<code>EscEcies_MAC_KEY_BYTES</code>	Length of the HMAC Key	Any unsigned 32-bit value	16U

Table 25: ECIES configuration

<i>Lib/src</i>	<i>Lib/inc</i>
<code>ecies.c</code>	<code>ecies.h</code>

Table 26: Associated files for ECIES

7.14 EdDsa / Ed25519

This module implements EdDSA in accordance to [RFC8032], [EdDsa], and [PureEdDSA].

Currently only ed25519 variants with SHA-512 are supported.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
eddsa.c	eddsa.h
_ptarithte25519.c	_ptarithte25519.h
_fearith.c	_fearith.h

Table 27: Associated files for EdDSA

7.15 Hash_DRBG

This module implements a pseudo-random number generator (PRNG) based on SHA-1, SHA-256, or SHA-512 in accordance to [HASH_DRBG]. The security strength of this PRNG is 128 bit to 256 bit (depending on the configuration) compared with a symmetric key algorithm. To reach this security strength it is necessary to use at least the desired security strength in bits as entropy input and a nonce with at least half of the desired security strength in bits extra entropy. Also some extra bytes of the nonce shall be used as personalization string, which makes every instance of this PRNG unique, for example the serial number of the host device is a good choice.

For more information about the definition of “security strength” see [NIST 800-57], section 5.6.1.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
EscHashDrbg_SECURITY_STRENGTH	The desired security strength in bits. This setting has no direct effect on the implementation, but it is used for some plausibility checks on other configuration options of the Hash-DRBG.	128, 192, 256	128
EscHashDrbg_ADJUSTABLE_OUT_LEN	If set to 1, the module generates random numbers of variable length. If set to 0, the module generates random numbers with a fixed length of EscHashDrbg_BLOCK_LEN bytes (depends on the used hash operation).	0 / 1	1
EscHashDrbg_RESEED_INTERVAL	Maximum number of PRNG requests before a reseed is enforced. Default value of 100000 is taken from example out of [HASH_DRBG]	Positive 32-bit number, i.e. not zero	100000U

EscHashDrbg _ENTROPY _LEN	Length of the input entropy array for function "Init" and "Reseed". For a security strength of 128 bit entropy must to be at least 16 bytes (at least security strength in bits divided by eight).	Positive 16-bit number, at least security strength in bits divided by 8.	16U
EscHashDrbg _NONCE_LEN	Length of the input nonce array for function "Reseed". For security strength of 128 bits the nonce must be at least 8 bytes.	Positive 16-bit number, at least security strength in bits divided by 16.	8U
EscHashDrbg _SHA_TYPE	Determines the hash algorithm. See values.	1 = SHA-1 224 = SHA-224 256 = SHA-256 384 = SHA-384 512 = SHA-512	256

Table 28: Hash_DRBG configuration

<i>Lib/src</i>	<i>Lib/inc</i>
hash_drbg.c	hash_drbg.h

Table 29: Associated files for Hash_DRBG

7.16 HMAC-RIPEMD-160

This module implements HMAC-RIPEMD-160 in accordance to [HMAC] and [RIPEMD].

Required module is RIPEMD-160.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
hmac_ripemd_160.c	hmac_ripemd160.h

Table 30: Associated files HMAC-RIPEMD-160

7.17 HMAC-SHA-1

This module implements HMAC-SHA-1 in accordance to [HMAC] and [SHA].

Required module is SHA-1.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
hmac_sha_1.c	hmac_sha_1.h

Table 31: Associated files for HMAC-SHA-1

7.18 HMAC-SHA-256

This module implements HMAC-SHA-256 in accordance to [HMAC] and [SHA].

Required module is SHA2-224 / SHA2-256.

No configuration parameters.

Supports SHA-224, SHA-256.

<i>Lib/src</i>	<i>Lib/inc</i>
hmac_sha_256.c	hmac_sha_256.h

Table 32: Associated files for HMAC-SHA-256

7.19 HMAC-SHA-512

This module implements HMAC-SHA-512 in accordance to [HMAC] and [SHA].

Required module is SHA2-384 / SHA2-512.

No configuration parameters.

Supports SHA2-384 and SHA2-512.

<i>Lib/src</i>	<i>Lib/inc</i>
hmac_sha_512.c	hmac_sha_512.h

Table 33: Associated files for HMAC-SHA-512

7.20 PKCS#1 RSASSA V1_5

This module implements PKCS#1 v2.2 signature generation and verification in accordance to [PKCS#1]. The supported functions are RSASSA-PKCS1-v1_5_Verify and RSASSA-PKCS1-v1_5_Sign.

The module RSA is required and the byte order of the long numbers is big-endian. Support for different hash algorithms can be enabled or disabled (see Table 34). Enabling an algorithm increases the code size.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
EscPkcs1RsaSsaV15_ MD2_ENABLED	Enable support for MD2 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ MD5_ENABLED	Enable support for MD5 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ SHA1_ENABLED	Enable support for SHA-1 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ SHA224_ENABLED	Enable support for SHA-224 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ SHA256_ENABLED	Enable support for SHA-256 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ SHA384_ENABLED	Enable support for SHA-384 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ SHA512_ENABLED	Enable support for SHA-512 algorithm.	0 / 1	1
EscPkcs1RsaSsaV15_ RIPEMD160_ENABLED	Enable support for RIPEMD-160 algorithm.	0 / 1	1

Table 34: PKCS#1 RSASSA V1_5 Configuration

<i>Lib/src</i>	<i>Lib/inc</i>
pkcs_rsassa_v15.c	pkcs_rsassa_v15.h
	pkcs1.h

Table 35: Associated files for PKCS#1 RSASSA V1_5

7.21 PKCS#1 RSASSA PSS

This module implements RSASSA PSS signature verification and generation in accordance to [PKCS#1]. The supported functions are RSASSA_sign and RSASSA_verify. The supported hash algorithms are SHA-1, SHA-224, SHA-256, SHA-384 and SHA-512.

The byte order of the long numbers is big-endian.

Required modules are RSA, a pseudo-random number generator such as CypcurLIB's Hash_DRBG implementation (see section 7.14), and subject to the configuration SHA-1, SHA-256 or SHA-512.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
EscPkcs1RsaSsaPss_ SHA1_ENABLED	Enable support for SHA-1 algorithm.	0 / 1	1

<code>EscPkcslRsaSsaPss_SHA224_ENABLED</code>	Enable support for SHA-224 algorithm.	0 / 1	1
<code>EscPkcslRsaSsaPss_SHA256_ENABLED</code>	Enable support for SHA-256 algorithm.	0 / 1	1
<code>EscPkcslRsaSsaPss_SHA384_ENABLED</code>	Enable support for SHA-384 algorithm.	0 / 1	1
<code>EscPkcslRsaSsaPss_SHA512_ENABLED</code>	Enable support for SHA-512 algorithm.	0 / 1	1
<code>EscPkcslRsaSsaPss_MAX_SALT_LEN</code>	Specify the maximum salt length which can be used.	Integer $\leq$ (size of modulus - digest length - 2)	Maximum digest length

Table 36: PKCS#1 RSASSA PSS configuration

<i>Lib/src</i>	<i>Lib/inc</i>
<code>pkcs_rsassa_pss.c</code>	<code>pkcs_rsassa_pss.h</code>
	<code>pkcs1.h</code>

Table 37: Associated files for PKCS#1 RSASSA PSS

7.22 PKCS#1 RSAES V1_5

This module implements RSAES-v1_5 asymmetric encryption and decryption in accordance to [PKCS#1]. The supported functions are RSAES-PKCS1-V1_5-ENCRYPT and RSAES-PKCS1-V1_5-DECRYPT.

The byte order of the long numbers is big-endian. Required modules are RSA and a pseudo-random number generator such as CypurLIB's Hash_DRBG implementation (see section 7.14).

<i>Lib/src</i>	<i>Lib/inc</i>
<code>pkcs_rsaes_v15.c</code>	<code>pkcs_rsaes_v15.h</code>
	<code>pkcs1.h</code>

Table 38: Associated files for PKCS#1 RSAES V1_5

7.23 PKCS#1 RSAES OAEP

This module implements RSAES OAEP asymmetric encryption and decryption in accordance to [PKCS#1]. The supported functions are RSAES-OAEP_encrypt and RSAES-OAEP_decrypt. The supported hash algorithms are SHA-1, SHA-224, SHA-256, SHA-384 and SHA-512.

The byte order of the long numbers is big-endian. Required modules are RSA, a pseudo-random number generator such as CypcurLIB's Hash_DRBG implementation (see section 7.14), and subject to the configuration SHA-1, SHA-256 or SHA-512.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
EscPkcs1RsaEsOaep_SHA1_ENABLED	Enable support for SHA-1 algorithm.	0 / 1	1
EscPkcs1RsaEsOaep_SHA224_ENABLED	Enable support for SHA-224 algorithm.	0 / 1	1
EscPkcs1RsaEsOaep_SHA256_ENABLED	Enable support for SHA-256 algorithm.	0 / 1	1
EscPkcs1RsaEsOaep_SHA384_ENABLED	Enable support for SHA-384 algorithm.	0 / 1	1
EscPkcs1RsaEsOaep_SHA512_ENABLED	Enable support for SHA-512 algorithm.	0 / 1	1

Table 39: PKCS#1 RSAES OAEP configuration

<i>Lib/src</i>	<i>Lib/inc</i>
pkcs_rsaes_oaep.c	pkcs_rsaes_oaep.h
	pkcs1.h

Table 40: Associated files for PKCS#1 RSAES OAEP

7.24 Poly1305

This module implements the Poly1305 one-time authenticator in accordance with [RFC7539].

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
poly1305.c	poly1305.h

Table 41: Associated files for Poly1305

7.25 RIPEMD-160

This module implements RIPEMD-160 in accordance to [RIPEMD].

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
ripemd_160.c	ripemd_160.h

Table 42: Associated files for RIPEMD-160

7.26 RSA

This module is the base module for all PKCS#1 modules and implements modular exponentiation for RSA encryption and decryption. It also supports the Chinese Remainder Theorem (CRT) which can significantly speed up the calculation, but which requires more information stored in the keys. Furthermore, the module implements the generation of RSA keys.



Warning!

This module only provides the raw exponentiation and does not apply / verify any padding! RSA is INSECURE if used without padding! Therefore, the functions of this module should not be used directly, but the functions of the PKCS#1 modules should be used instead!

The exponentiation can be significantly accelerated by enabling `Esc_HAS_UINT64`. Algorithm optimizations can be configured in the header file `rsa_cfg.h` which also includes hints which variant of the algorithms should be used for your application. Table 43 gives an overview of the configuration options for the RSA module. The long numbers are expected to be in the big-endian byte order. The key generation requires a pseudo-random number generator such as CyclesLIB's Hash_DRBG implementation (see section 7.14).

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscRsa_KEY_BITS_MAX</code>	Maximum supported bit length of the RSA module	1024, 1536, 2048, 3072, 4096	1024
<code>EscRsa_ENABLE_GENRSA</code>	Enable the RSA key generation	0 / 1	0
<code>EscRsa_ENABLE_CRT</code>	Enable the RSA exponentiation with CRT	0 / 1	0
<code>EscRsa_PRIMETEST_ITER</code>	Number of iterations of the Miller-Rabin primality test. Only used during key generation.	Any positive 32-bit integer	5U
<code>EscRsa_USE_MONTGOM_MUL</code>	Enable the Montgomery Reduction	0 / 1	Speed: 1 Size: 0
<code>EscRsa_USE_SLIDING_WINDOW</code>	Enable for using the Sliding Window algorithm	0 / 1	Speed: 0 Size: 0
<code>EscRsa_SLICING_ENABLED</code>	Enable to use the sliced variant.	0 / 1	0

Table 43: RSA configuration

<i>Lib/src</i>	<i>Lib/inc</i>
rsa.c	rsa.h
_rsa.c	_rsa.h
	rsa_cfg.h
	modexp.h
Optional files (depending on your use case and configuration)	
modexp_s0.c	modexp_s0.h
modexp_s1.c	modexp_s1.h

Table 44: Associated files for RSA

7.27 SHA-1

This module implements SHA-1 in accordance to [SHA].



Warning!

The MD5 hash algorithm is INSECURE and it should only be used if compatibility with legacy applications is required! Otherwise, a secure hash algorithm (e.g. SHA2) should be used instead.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
sha_1.c	sha_1.h

Table 45: Associated files for SHA-1

7.28 SHA2-224 and SHA2-256

This module implements SHA-224 and SHA-256 in accordance to [SHA]. Both algorithms are integrated in source file sha_256.c.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
sha_256.c	sha_256.h

Table 46: Associated files for SHA2-256

7.29 SHA2-384 and SHA2-512

This module implements SHA-384 and SHA-512 in accordance to [SHA]. Both algorithms are integrated in source file `sha_512.c`.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
sha_512.c	sha_512.h

Table 47: Associated files for SHA2-512

7.30 SHA3

This module implements SHA3-224, SHA3-256, SHA3-384 and SHA3-512 in accordance to [SHA3]. The algorithms are implemented in source file `sha_3.c`.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
sha_3.c	sha_3.h

Table 48: Associated files for SHA3

7.31 SHE-MP

This module implements the AES-128 based Miyaguchi-Preneel compression function and key derivation as specified in [SHE]. The algorithms are implemented in source file `she_mp.c`.

No configuration parameters.

<i>Lib/src</i>	<i>Lib/inc</i>
she_mp.c	she_mp.h

Table 49: Associated files for SHE-MP

7.32 Whirlpool

This module implements the Whirlpool hash function in accordance to [WHIRLPOOL].

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>
<code>EscWhirlpool_8BIT_OPTIMIZATION</code>	Increases speed significantly on costs of a lookup-table of 256 bytes size.	0 / 1	Speed: 1 Size: 0

Table 50: Whirlpool configuration

<i>Lib/src</i>	<i>Lib/inc</i>
<code>whirlpool.c</code>	<code>whirlpool.h</code>

Table 51: Associated files for Whirlpool

7.33 X.509

This module implements X.509 [X509] certificate parsing and certificate verification. The following signature algorithms are supported:

- `sha1WithRSAEncryption`
- `sha224WithRSAEncryption`
- `sha256WithRSAEncryption`
- `sha384WithRSAEncryption`
- `sha512WithRSAEncryption`
- `ecdsa-with-SHA1`
- `ecdsa-with-SHA224`
- `ecdsa-with-SHA256`
- `ecdsa-with-SHA384`
- `ecdsa-with-SHA512`

The interface of the X.509 parser is located in the file `x509.h`, which also contains the configuration options shown in Table 52. The options for enabling the algorithms have an effect on both the signature and the public key of the algorithm. For example, parsing and verifying an RSA-PKCS#1v1.5 signed certificate containing an ECC public key would require enabling both `EscX509_ALGORITHM_RSA_PKCS1V15_ENABLED` and `EscX509_ALGORITHM_ECDSA_ENABLED`. The parser supports the following extensions:

- `basicConstraints`
- `keyUsage`

Other extensions are ignored by the parser unless they are flagged as critical in the certificate. Unknown critical extensions lead to the rejection of the certificate. The parser supports X.509 certificates in version 1, 2 and 3. The following attributes of issuer and subject are supported:

- Common name (CN)
- Locality (L)

- State or province (ST)
- Organization (O)
- Organizational Unit (OU)
- Country (C)

Each attribute must be present at most once in the certificate in order to query the attribute using the parser's API.

The parser is limited to the configured RSA key size (see section 7.26) with public exponents smaller than 2^{32} bits and the configured ECC curves (see section 7.11).

The certificate must be encoded in the DER format to be processed by the parser. Other encodings (e.g. PEM) are not supported. The DER parser is always required. Additional modules may be required depending on the configuration as shown in Table 52.

<i>Preprocessor Symbol</i>	<i>Description</i>	<i>Values</i>	<i>Default</i>	<i>Additional Modules Required</i>
EscX509_PARSE_INFO	If defined, all supported x.509 attributes are readout. Otherwise, only the public key, the version, the signature and extensions are processed. The complete parsing increases ROM and RAM-usage due to the additional required DER functions.	0 / 1	1	
EscX509_ALGORITHM_RSA_PKCS1V15_ENABLED	Enable support for parsing RSA PKCS#1-v1.5 signatures and public keys.	0 / 1	1	RSA, PKCS#1 RSASSA V1_5
EscX509_ALGORITHM_RSA_PKCS1PSS_ENABLED	Enable support for parsing RSA PKCS#1-PSS signatures and public keys.	0 / 1	0	RSA, PKCS#1 RSASSA PSS
EscX509_ALGORITHM_ECDSA_ENABLED	Enable support for parsing ECDSA signatures and ECC public keys.	0 / 1	0	ECC
EscX509_HASH_SHA1_ENABLED	Enable support for signatures using SHA-1 as hash function.	0 / 1	0	SHA-1
EscX509_HASH_SHA224_ENABLED	Enable support for signatures using SHA2-224 as hash function.	0 / 1	1	SHA2-224/256
EscX509_HASH_SHA256_ENABLED	Enable support for signatures using SHA2-256 as hash function.	0 / 1	1	SHA2-224/256
EscX509_HASH_SHA384_ENABLED	Enable support for signatures using SHA2-384 as hash function.	0 / 1	0	SHA2-384/512
EscX509_HASH_SHA512_ENABLED	Enable support for signatures using SHA2-512 as hash function.	0 / 1	0	SHA2-384/512

Table 52: X.509 configuration

<i>Lib/src</i>	<i>Lib/inc</i>
x509.c	x509.h
<i>Optional files (depending on your use case and configuration)</i>	
x509_ecc.c	x509_ecc.h
x509_rsa.c	x509_rsa.h

Table 53: Associated files for X.509

8 References

AES	FIPS 197 Federal Information Processing Standards Publication 197, ADVANCED ENCRYPTION STANDARD (AES), 26. November 2001 http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf
ASN	Abstract Syntax Notation 1 http://www.asn1.org/
BLOWFISH	Blowfish block cipher https://www.schneier.com/blowfish.html
BRAINPOOL RFC7539	ECC Brainpool Standard Curves and Curve Generation, Version 1.0, 19.10.2005 ChaCha20 and Poly1305 for IETF Protocols https://tools.ietf.org/html/rfc7539
CMAC	NIST Special Publication 800-38B: Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication, May 2005 http://csrc.nist.gov/publications/nistpubs/800-38B/SP_800-38B.pdf
CRC32	IEEE 802.3 Cyclic Redundancy Check, (v1.0) 23. March 2001
ECIES	IEEE Std 1363a™-2004
EdDsa	Bernstein et al.: High-speed high-security signatures (2011) https://ed25519.cr.yp.to/ed25519-20110705.pdf
RFC8032	Edwards-Curve Digital Signature Algorithm (EdDSA) https://tools.ietf.org/html/rfc8032
DER	ITU-T x.690 (Distinguished Encoding Rules for ASN.1) http://www.itu.int/ITU-T/studygroups/com17/languages/X.690-0207.pdf
DES	Data Encryption Standard (DES), FIPS PUB 46-3, 1999 October 25 http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf
HASH_DRBG	NIST Special Publication 800-90: Recommendation for Random Number Generation Using Deterministic Random Bit Generators (Revised), March 2007 http://csrc.nist.gov/publications/nistpubs/800-90/SP800-90revised_March2007.pdf
HMAC	RFC 2104: HMAC: Keyed-Hashing for Message Authentication http://www.ietf.org/rfc/rfc2104.txt
NIST 800-56	NIST Special Publication 800-57A Revision 2: Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography, May 2013 http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Ar2.pdf
NIST 800-57	NIST Special Publication 800-57: Recommendation for Key Management – Part1: General (Revised), March 2007 http://csrc.nist.gov/publications/nistpubs/800-57/sp800-57-Part1-revised2_Mar08-2007.pdf
PKCS#1	PKCS #1 v2.1: RSA Cryptography Standard, RSA Laboratories, 14. Juni 2002 http://www.rsasecurity.com/rsalabs/node.asp?id=2125
PureEdDSA	Bernstein et al.: EdDSA for more curves (2015) https://ed25519.cr.yp.to/eddsa-20150704.pdf
RIPEMD	ISO/IEC FDIS 10118-3 Final Draft 2003 Hash Standards
SEC1	STANDARDS FOR EFFICIENT CRYPTOGRAPHY, SEC 1: Elliptic Curve Cryptography, 20. September 2000, Version 1.0

ESCRYPT

SEC2	STANDARDS FOR EFFICIENT CRYPTOGRAPHY, SEC 2: Recommended Elliptic Curve Domain Parameters, 20. September 2000, Version 1.0
SHA	FIPS-180-2 http://csrc.nist.gov/publications/fips/fips180-2/fips180-2.pdf
SHA3	FIPS PUB 202 http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
SHE	SHE – Secure Hardware Extension, Functional Specification 01. April 2009, Version 1.1, Revision 439
WHIRLPOOL	ISO/IEC FDIS 10118-3 Final Draft 2004 Hash Standards
X509	ITU-T Recommendation X.509 http://www.itu.int/rec/dologin_pub.asp?lang=e&id=T-REC-X.509-200508-!!!PDF-E&type=items

9 **ESCRYPT Contact Addresses**

ESCRYPT HQ

ESCRYPT GmbH

Wittener Str. 45

44789 Bochum

Germany

Phone: +49 234 43870-200

Fax: +49 234 43870-211

WWW: www.escrypt.com

ESCRYPT Subsidiaries

For details of your local sales office as well as your local technical support team, take a look at the ESCRYPT website:

ESCRYPT subsidiaries

WWW: www.escrypt.com/company/contact